

Project Title: Model Comparison for Travel Time Prediction (Team B)

Objective:

The goal of this project is to evaluate and compare the predictive performance of two regression models — Lasso Regression and Random Forest — for estimating actual travel time using a feature-engineered dataset. Key tasks include preprocessing, model training, hyperparameter tuning, and performance evaluation.

1. Data Loading and Preparation

Three datasets were used:

- - `feature\_engineered\_dataset.csv`
- - `scaled\_features\_dataset.csv`
- - `pca\_reduced\_dataset.csv`

From these, three sets of features were extracted:

- - **Main Features:** Raw features (for Random Forest)
- - **Scaled Features:** Standardized features (for Lasso)
- - **PCA Features:** Reduced dimensionality features (for potential future use)

The target variable is `actual\_time` (or its standardized version `actual\_time\_standard`). Train-test split was done with a 80:20 ratio and a fixed random seed.

2. Lasso Regression (LassoCV)

- - **Preprocessing:**
 - Only numerical features were used.
 - Removed infinite and missing values.
 - Dropped irrelevant or potentially leakage-inducing columns.
 - Features were standardized using `StandardScaler`.

- **- Modeling:**

- Used `LassoCV` with cross-validation over alpha values [0.05, 0.08, 0.1, 0.12, 0.15, 0.2].

- **- Evaluation Metrics:**

- R^2 Score
- Root Mean Squared Error (RMSE)
- Mean Absolute Error (MAE)
- Mean Absolute Percentage Error (MAPE)

- **- Results:**

- Best Alpha: *Selected via cross-validation*
- Train R^2 : High, indicating good fit
- Test R^2 : Reasonable, suggesting generalization

3. Random Forest Regression

- **- Preprocessing:**

- Handled categorical and numerical columns using a `ColumnTransformer` pipeline.
- Imputation strategies: mean for numerical, constant for categorical.
- One-hot encoding for categorical columns.

- **- Modeling:**

- Random Forest with `RandomizedSearchCV` for hyperparameter tuning.
- Trained on 30% sample for search, then retrained on full training set using best parameters.

- **- Hyperparameter Search Space:**

- `n_estimators`: [50, 100]

- max_depth: [10, None]
- min_samples_split: [2, 5]
- min_samples_leaf: [1, 2]
- max_features: ["sqrt"]
- bootstrap: [True]

- - **Feature Importance:**

- Top 20 most influential features plotted using a horizontal bar chart.

- - **Evaluation Metrics:**

- R² Score
- RMSE
- MAE
- MAPE

4. Model Comparison Summary

The results were compiled in a comparison dataframe:

	LassoCV (No Leakage)	RandomForest
Best Alpha	0.050000	NaN
Train R²	0.981851	0.864240
Test R²	0.982706	0.859886
Train RMSE	81.688854	222.439885
Test RMSE	78.988418	225.229976

LassoCV (No Leakage)	RandomForest	
Train MAE	38.237644	117.162186
Test MAE	38.382639	119.349382
Train MAPE	40.475763	71.168778
Test MAPE	42.204491	73.020186

Conclusion:

Best Model: LassoCV

Why LassoCV wins:

- Much higher **R^2 scores** (better fit on both train and test sets).
- Significantly lower **RMSE, MAE, and MAPE**, meaning it's **more accurate** and **consistent**.
- Model is also simpler and interpretable (due to regularization/shrinkage).